Setup

- Go to the DSA4E folder.
- Under labs, create a folder named lab0.
- Inside lab0, create the following folders:

build
src

In lab0 folder create this file **:** CMakeLists.txt

You do not need to create the bin folder; it will be created automatically when the executable is generated.
You also do not need to create the data folder, since files are not used in this lab.

- Under src, create a folder named your_andrewid.
- Inside your your_andrewid folder, create the following three files:

interface.h
implementation.cpp
main.cpp

The content of the CMakeLists.txt file is the same as assignment0.
Simply replace every occurrence of assignment0 with lab0.

Problem 1: Age Classification

Given Function Signature
void classifyAge();

Write a function that:
- Reads a user's age.
- Displays:
  "Invalid age" if age < 0
  "Child" if age is 0-12
  "Teenager" if age is 13-19
  "Adult" if age ≥ 20
- Use if / else statements.

Problem 2: Display Numbers from 1 to N_MAX

Given Function Signature
void displayNumbersUpToMax();

Write a function that:
- Displays all numbers from 1 to N_MAX, inclusive.

- Each number should be printed on the same line separated by spaces.
- Use a for loop.

Problem 3: Display All Odd Numbers in an Interval

Given Function Signature

void displayOddNumbers(int start, int end);

- Write a function that:
- Displays all odd numbers between start and end (inclusive).

Assume start <= end.

Use:

- A loop
- The modulo operator %

Problem 4: Sum of Numbers in an Interval

Given Function Signature

int sumInterval(int start, int end);

Write a function that:

Returns the sum of all integers between start and end (inclusive).
The function must return the result (not print it).

Example:
sumInterval(1, 5) -> 15

Problem 5: Display Prime Numbers in an Interval

Given Function Signature

void displayPrimeNumbers(int start, int end);

Definition

A prime number is a number:

- Greater than 1
- Divisible only by 1 and itself

Problem Statement

Write a function that:
Displays all prime numbers between start and end (inclusive).

Nota Bene

At the end of the lab your interface.h should look like this :

```cpp
#ifndef INTERFACE_H
#define INTERFACE_H

const int N_MAX = 50;

// Problem 1
void classifyAge();

// Problem 2
void displayNumbersUpToMax();

// Problem 3
void displayOddNumbers(int start, int end);

// Problem 4
int sumInterval(int start, int end);

// Problem 5
void displayPrimeNumbers(int start, int end);

#endif

```

Your implementation.cpp file :

```cpp
#include <iostream>
#include "interface.h"

using namespace std;

// Problem 1
void classifyAge(){
    cout << "Chop rice !!!" << endl;
}

// Problem 2
void displayNumbersUpToMax(){
    cout << "Chop rice !!!" << endl;
}

// Problem 3
void displayOddNumbers(int start, int end){
    cout << "Chop rice !!!" << endl;
}

// Problem 4
int sumInterval(int start, int end){
    cout << "Chop rice !!!" << endl;
}

// Problem 5
void displayPrimeNumbers(int start, int end){
    cout << "Chop rice !!!" << endl;
}
```

Your main.cpp file should look like this:

```cpp
#include <iostream>
#include "interface.h"

using namespace std;

int main() {

    // Problem 1
    // classifyAge();

    // Problem 2
    // displayNumbersUpToMax();

    // Problem 3
    // displayOddNumbers(1, 20);

    // Problem 4
    // int result = sumInterval(1, 10);
    // cout << "Sum: " << result << endl;

    // Problem 5
    // displayPrimeNumbers(1, 50);

    return 0;
}
```